2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER) | 979-8-3503-3066-3/24/$31.00 ©2024 IEEE | DOI: 10.1109/SANER60148.2024.00095

FLAKYRANK: Predicting Flaky Tests Using Augmented Learning to Rank

Jiaguo Wang[1], Yan Lei[1,2*], Maojin Li[1], Guanyu Ren[3],Huan Xie[1],Shifeng Jin[1],Junchao Li[1],Jian Hu[1]
[1] School of Big Data & Software Engineering, Chongqing University, Chongqing, China
[2]Peng Cheng Laboratory, Shenzhen, China
[3]University College London, London, United Kingdom
wangjg@stu.cqu.edu.cn, yanlei@cqu.edu.cn,towellee@cqu.edu.cn,18561857186@163.com,
huanxie@cqu.edu.cn, feng_poi@126.com,lijunchao@stu.cqu.edu.cn,jianhu@cqu.edu.cn

***Abstract*—The ideal principle of software testing is that test results ought to be deterministic: a test failure indicates the presence of a software bug, while a test success suggests the absence of a bug. Nevertheless, flaky tests break the principle. Flaky tests yield inconsistent results when executed repeatedly under the same conditions. The most straightforward approach runs the tests multiple times to predict flaky tests whereas it is highly time-consuming. Many researchers have proposed efficient approaches to reduce the cost, e.g., recent approaches leverage machine learning techniques for the prediction of flaky tests. However, traditional machine learning primarily focuses on predicting specific instances, which is not conducive to identify flaky tests across an entire project.**

Therefore, we propose FLAKYRANK, a ranking framework based on augmented learning to rank to predict flaky tests. The insight is that learning to rank, as compared with traditional machine learning, not only concentrates on individual samples but also optimizes the overall ranking. Since flaky tests constitute a small proportion of the dataset (i.e., approximately 3.6% of the total tests), we utilize generative adversarial networks to generate some synthetic flaky tests to augment the dataset. Based on the augmented dataset, FLAKYRANK treats predicting flaky tests as an information retrieval task, where newly detected flaky tests and test cases serve as queries and documents, respectively. For each newly detected flaky test (i.e., query), FLAKYRANK combines multiple relevant features into a learning to rank model to predict flaky tests candidate tests. We conduct large-scale experiments on different learning to rank models, and the results show that FLAKYRANK with the LambdaMART algorithm yields the best performance. In addition, the experimental results on 23 benchmark projects show that FLAKYRANK outperforms the state-of-the-art predictors.

***Index Terms*—Software testing, flaky tests, learning to rank, generative adversarial networks**

I. INTRODUCTION

To ensure the correctness, completeness, security, and quality of software, software testing [1]–[4] requires an adequate number of effective tests to assist practitioners in discovering software defects in the software product. However, the presence of unstable test cases in test can have significant impacts on both assisting practitioners and controlling the development costs of software products. In particular, one potential issue is the phenomenon where different test results occur in the same test scenario and environment. Such tests, known as flaky tests, show different test results (i.e., both passes and failures) despite no changes to the code or the test itself [5]–[7].

Flaky tests [8]–[10] are a common issue in real-world software projects, which can be observed from open-source small projects to large productions released by companies such as Google and Microsoft [11]. Memon et al. [11] has found that 41% of tests at Google have exhibited non-unique test outcomes. Flaky tests bring a detrimental impact on the reliability and efficiency of tests [8], [12]. Failures of flaky tests are difficult to reproduce, which consumes much time of developers in diagnosing issues that may not be real defects and reduces the value of software testing in finding real faults [13]. The uncertainty associated with flaky tests can erode the trust of developers, potentially causing them to overlook real faults, with serious consequences for the development process [8]. Therefore, flaky tests can impact the efficiency of testing. In software development, developers use continuous integration to automate the process of running tests and merging changes [14]. However, when developers merges their changes, the appearance of flaky tests makes developers no longer trust the previous test results. In order to fix the failure of tests caused by flaky tests, developers need to spend more time to debug. In other words, the instability of flaky tests leads to false defects alarms, degrading project development efficiency [15], [16].

Existing approaches for automatically predicting flaky tests [7] include those static ones that analyze them statically without running the test suite [8] and those dynamic ones that require the execution of test suites to extract dynamic information. For those static approaches, they do not require test executions, e.g., utilizing static code analysis [17], examining test structures [8], and leveraging test-related vocabulary [18] based on the frequency of identifiers and keywords to predict flaky tests. Due to the lack of dynamic information, the static approaches may produce more false positives and restrict the performance of flaky test detection. Thus, many researchers explore dynamic approaches using dynamic information. For dynamic approaches, the most straightforward one for detecting flaky tests is to repeatedly execute test cases and observe whether the test results are consistent or not [13], [19], [20]. Nonetheless, this approach is inherently time-intensive and presently serves as the benchmark for flaky

*Yan Lei is the corresponding author.

2640-7574/24/$31.00 ©2024 IEEE
DOI 10.1109/SANER60148.2024.00095

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

test identification [8].

In addition, the dynamic approaches that require test executions also consider specific properties [21]–[23], such as change in test coverage [24], to identify flaky tests. Some recent approaches also employ machine learning models [17], [25], [26] or large language models [27], [28] to dynamically identify flaky tests and show promising results. However, existing methods still face several challenges: (1) In the domain of datasets, there is a scarcity of flaky tests in the complete datasets, which can affect model training. (2) Traditional machine learning detection methods primarily target specific types of flaky tests, and their accuracy is expected to be improved. (3) Large language models of flaky tests have room for development in terms of training duration and model deployment.

To bridge the gap, we propose **FLAKYRANK**: an approach predicting **Flaky** tests using augmented learning to **Rank**. FLAKYRANK have two main components: data augmentation and ranking flaky tests. Specifically, data augmentation, using Generative Adversarial Networks (GANs) [29]–[31] to generate synthetic flaky tests to augment the dataset, aims to overcome the impact of imbalanced datasets. Moreover, learning to rank is employed to address the issue of weight matching for different features, and optimize overall relevance for ranking. Based on the augmented dataset, FLAKYRANK can adequately integrate the behavior features of a test program to identify flaky tests using a learning to rank model [32], and rank those suspicious tests of being flaky. In addition, compared to large language models, FLAKYRANK is not limited by a program scale to predict flaky tests and easier to employ.

To evaluate FLAKYRANK, we conduct large-scale experiments on the typical dataset, the FlakeFlagger dataset, which comprises 24 open-source projects from GitHub (one project without flaky tests), with each test repeated 10 thousand times [26]. The results show that FLAKYRANK with the learning to rank model (i.e., the LambdaMART algorithm belonging to *listwise* approach) yields the best performance, and also demonstrate that FLAKYRANK outperforms the state-of-the-art flaky test predictors.

The main contributions of this paper can be summarized as follows:

- We propose an augmented learning to rank approach combining data augmentation with learning to rank to predict flaky tests.
- We propose a GAN-based augmentation approach specialized for improving flaky test prediction.
- We conduct large-scale experiments on 23 benchmark projects to evaluate FLAKYRANK and the results show that FLAKYRANK is effective to predict flaky tests.
- Public available tool [1].

The structure of the rest paper is organized as follows. Section II depicts the necessary background. Section III depicts our approach FLAKYRANK. Section IV shows our large-scale experiments and the results. Section V discusses the impact of data augmentation and the threats to the validity. Section VI introduces the related work. Section VII concludes the whole study and mentions future work.

[1]https://github.com/indentityuti/FLAKYRANK

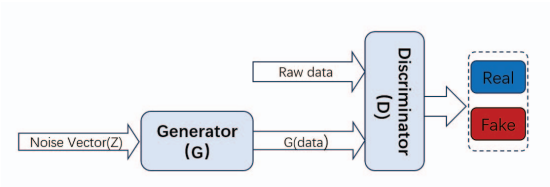


Fig. 1. The framework of GANs.

II. BACKGROUND

A. Generative Adversarial Networks

Generative Adversarial Networks (GANs) [29], [33] are a type of deep learning model used for data generation. GANs consist of two neural networks, one is the generator and the other is the discriminator, which are trained together through competitive training. The goal of the generator is to produce data that is difficult to distinguish from real data, while the discriminator aims to differentiate between real and generated data. This adversarial training process helps enhance the ability to create realistic data. As showed in Fig. 1, GANs consist of two components: the generator (G) and the discriminator (D) [34]. G generates synthetic data by taking random noise or latent representations as input. G continuously refines its parameters to improve the generated data, with the objective of making it difficult for D to distinguish between the generated data and real data. D classifies data, discerning between real and fake. D evaluates both data generated by G and authentic data with the goal of accurately distinguishing between them.

There exists a competitive relationship between G and D, with G attempting to create increasingly realistic data to deceive D, while D works to improve its classification accuracy. The adversarial training process drives G to continuously enhance the quality of the generated data, ultimately aiming to generate synthetic data that is nearly indistinguishable from real data.

The fundamental objective function of GANs is shown in Eq. 1.

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim pdata(x)}\left[logD(x)\right] + \mathbb{E}_{z\sim pz(z)}\left[log(1-D(G(z)))\right] \quad (1)$$

Eq. 1 captures the adversarial training process between these two components. It minimizes $V(D,G)$, where D seeks to maximize this value and G aims to minimize it. $\mathbb{E}_{x\sim pdata(x)}\left[logD(x)\right]$ represents the expected value over real data samples, where x follows the distribution $pdata(x)$. It measures how well the discriminator can correctly classify real data as real. $\mathbb{E}_{z\sim pz(z)}\left[log(1-D(G(z)))\right]$ represents the expected value over samples generated by G, where z follows the distribution $pz(z)$. It measures how well the discriminator can correctly classify generated data as fake.

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

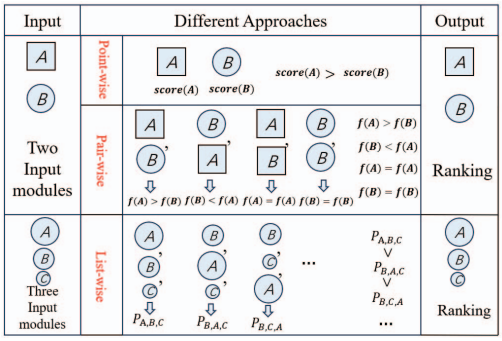


Fig. 2. The overview of the three types of Learning to Rank methods: *pointwise*, *pairwise* and *listwise*.

G strives to generate data that is indistinguishable from real data ($log(1 - D(G(z)))$ should be close to 0), while D aims to improve its ability to distinguish real from generated data ($logD(x)$ should be close to 1) [35]. The adversarial training dynamic continues iteratively, with both components pushing each other to improve. Eventually, a Nash equilibrium [36] is reached. G generates high-quality synthetic data that is difficult for D to distinguish from real data, leading to the desired outcome of GANs: the generation of realistic data.

B. Learning to Rank

Fig. 2 shows the basic idea of the Learning to Rank (LTR) methods, from which we can see that the sorting process for *pointwise*, *pairwise*, and *listwise* of LTR methods. A, B, and C are input documents in Fig. 2. For *pointwise*: square A is more relevant to the query than prototype B. For *pairwise* and *listwise*: the larger the area of the prototype, the more relevant it is to the query [37], which corresponds to a relevance ranking of A, B, C. Detailed introductions to LTR methods are presented below.

1) Pointwise Approach: The *pointwise* approach deals with individual documents. FLAKYRANK can integrate multiple machine learning methods for ranking. Fig. 2 shows the fundamental principles of the *pointwise* method ranking, which demonstrates the ranking process [38] for two documents (i.e., square A and circle B). Assuming there is a query, the *pointwise* approach predicts the likelihood of module A as Score(A) and module B as Score(B), if Score(A) $>$ Score(B), then the predicted ranking of A and B is denoted as A $>$ B.

2) Pairwise Approach: The *pairwise* approach compares document pairs instead of directly predicting ranking scores for individual documents. It focuses on determining the relative ranking of document pairs for a given query, transforming the task into a binary classification problem. The *pairwise* approach is to predict which document should hold a higher position in the ranking. Positive feedback is given when the prediction matches the actual relative ranking, while negative feedback is given when it does not. The *pairwise* learning approach helps train models to capture the relative importance between different documents, rather than merely predicting the absolute scores for individual documents. By repeatedly comparing document pairs, the model can learn more refined ranking rules and feature weights, thereby improving ranking performance.

As shown in Fig. 2, assuming the *pairwise* approach predicts that module A is flaky while module B is not (i.e., f(A)>f(B)), we classify module A as flaky and module B as non-flaky. Conversely, if f(A) = f(B), both modules are classified as non-flaky, resulting in the predicted ranking of A>B.

3) Listwise Approach: The *listwise* approach [39], [40] directly optimizes the entire document ranking list, rather than just focus on document pairs (*pairwise*) or individual document ranking scores (*pointwise*). Which allows the *listwise* approach to better consider the collective impact of the entire ranking list, resulting in more superior ranking performance. The *listwise* loss functions evaluate between the model-generated ranking list and the actual result. The *listwise* loss functions can be computed as Eq. 2, where ν_i is sample document, $r(\nu_i)$ and $p(\nu_i)$ represent real label and ranking result of the sample ν_i, respectively.

$$\mathbb{L}_{list} = -\sum_{i=1}^{n} r(\nu_i) \log(p(\nu_i)) \tag{2}$$

As shown in Fig. 2, assuming that the *listwise* approach predicts that the ranking list $P_{A,B,C}$ yields the best performance measure among all possible ranking lists (i.e., $P_{A,B,C}$; $P_{A,C,B}$; $P_{B,A,C}$ etc.), the predicted ranking of A, B, and C will be A $>$ B $>$ C.

III. APPROACH

In this section, we introduce the basic idea of FLAKYRANK, a ranking framework for predicting flaky tests, which transforms the identification of flaky tests into a ranking problem. We will introduce the data processing including data augmentation in FLAKYRANK (section III-A) and depict how FLAKYRANK predict flaky tests (section III-B). Fig. 3 shows the framework of FLAKYRANK. As shown in Fig. 3, FLAKYRANK collects behavior features from the dataset through static analysis and running tests. Then, FLAKYRANK employs GANs to obtain an augmented dataset (i.e., a class-balanced dataset) for training. After obtaining the augmented dataset, FLAKYRANK adjusts the data paradigm for training based on the integrated LTR methods (i.e., *pointwise*, *pairwise*, and *listwise*). FLAKYRANK considers the prediction of flaky tests as a query, and the pre-structured data as documents. Finally, FLAKYRANK predict flaky tests via the ranking list of candidate flaky tests.

A. Data Processing

Feature Representation. Feature representation refers to the extracted features for each test from the FlakeFlagger dataset through static analysis and running tests. Motivated by prior research on test flakiness [8], we utilize 23 features that might be associated with flaky tests, including test smells, test case complexity, and hard-coded values. Some of these features

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

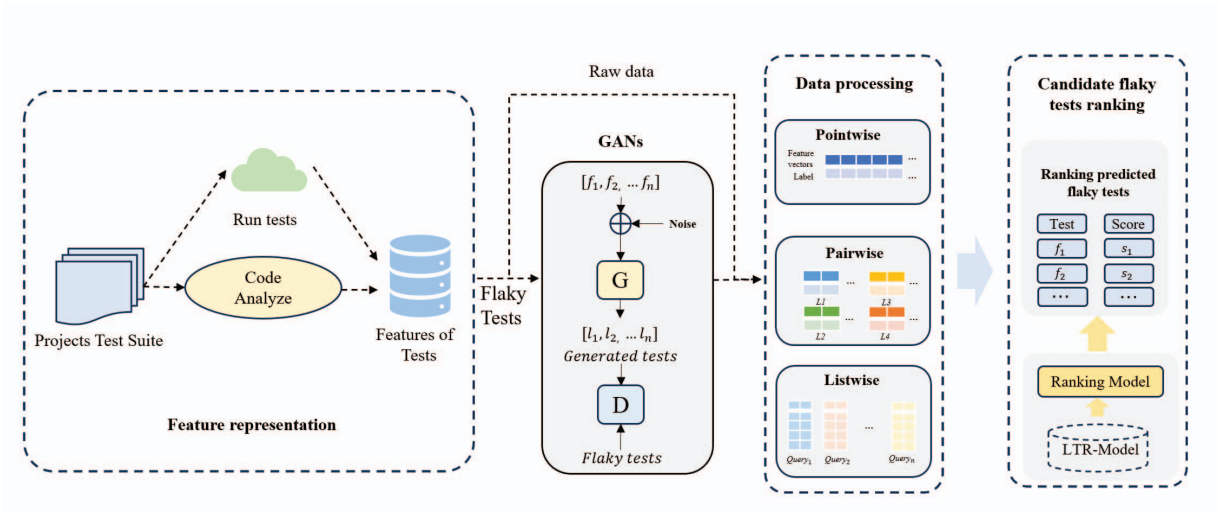


Fig. 3. The architecture of FLAKYRANK.

TABLE I
THE DETAILED DECRIPTION OF TEST SMELLS.

Test Smell	Desription
Assertion Roulette	Indicates that a test contains multiple assertions, possibly implying a lack of test focus
Conditional Logic	Identifies tests that contain a conditional if-statement within the test method body
Eager Testing	Signifies tests that exercise more than one method of the tested object
Fire and Forget	Points out tests that launch background threads or tasks
Indirect Testing	Marks tests that interact with the object under test via an intermediary
Mystery Guest	Flags tests that access external resources, potentially introducing hidden dependencies
Resource Optimism	Highlights tests that access external resources without checking their availability
Test Run War	Indicates tests that allocate a file or resource that might be used by other tests

are derived from general investigations into the root causes of flaky tests [12], [41], while others originate from practices involving various static analysis techniques for assessing automated unit testing [42]. Test smells refer to sub-optimal design or implementation decisions made by developers during the creation of test cases. While not necessarily causing functional issues in the test, test smells can introduce elements of instability, making the test more likely to exhibit flakiness. Test smells have been used as criteria for categorizing whether a test case exhibits flakiness. Alshammari et al. [43] and Camara et al. [44] discovered a correlation between test smells and test flakiness. They proposed methods that utilize test smells as features for predicting flaky tests. We used the test smells detailed in Table I as features for predicting flaky tests.

Other behavior features of tests are also related to predicting flaky tests, and a detailed description of these features is provided in Table II.

Data Augmentation. Data augmentation increases the quantity of small samples to balance the dataset, achieving this by generating synthetic flaky tests using generative adversarial networks to prepare the data for model training [45]. The proportion of flaky tests in FlakeFlagger dataset is minimal, i.e., approximately 3.6%. Some projects (e.g., ninja and assertj-core) have only a single flaky test. Data imbalance can significantly impact the accuracy of model, which motivates us to employ GANs to construct an augmented dataset (i.e., a class-balanced dataset). As shown in Fig. 3, flaky test vector $[f_1, f_2 ... f_n]$, is inputted into the GANs model, and the model outputs the synthetic flaky test vector $[l_1, l_2 ... l_n]$, where f, l, and n repersent the feature value of the raw and synthetic vectors, the number of features, respectively.

The training procedure for GANs is as follows: the flaky tests vectors $[f_1, f_2 ... f_n]$ are dealed with some noise Z, and then inputted into the generating network G [46]. G generates the synthetic vector $[l_1, l_2 ... l_n]$, which is inputted into the discriminating network D, and every systhetic vector is assigned a score. Real flaky tests are also inputted into D and set to be a score value of 1. During the initial stages of training, the generated flaky tests are given a score value of 0, and G is relatively weak. The difference between these two score values is continuously fine-tuned through loss back-propagation, and gradually strengthening G until the point where D can no longer distinguish between raw data and synthesized flaky tests. After training, GANs generate synthetic flaky tests in the same quantity as the non-flaky tests samples in the dataset until a class-balanced dataset is obtained, i.e., an augmented dataset. FLAKYRANK then trains the ranking model using the augmented data.

B. Predicting Flaky Tests

FLAKYRANK is a ranking framework based on LTR methods, and it transforms the prediction of flaky tests into a ranking problem. FLAKYRANK can integrate multiple LTR algorithms. We employed eight classical LTR algorithms for different supervised learning approaches:

- Four different supervised learning algorithms: Random Forest (RF) [47], Support Vector Machine (SVM) [48], Multilayer Perceptron (MLP) [49], and K-Nearest Neighbor (KNN) [50] for the *pointwise* approach.
- Two classical *pairwise* algorithms: RankNet [51] and SVMRank [52] for the *pairwise* approach.

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DESCRIPTION OF OTHER BEHAVIOR FEATURES.

Feature	Desription
Test Lines of Code	It represents the count of lines of code within the body of the test method
Number of Assertions	This indicates the quantity of assertions verified by the test
Execution Time	It refers to the duration taken for the test to execute
Source Covered Lines	This denotes the number of lines covered by each test, specifically focusing on production code
Covered Lines	It signifies the overall count of lines of code that are encompassed by the test
Source Covered Classes	This represents the total number of production classes covered by each test
External Libraries	It indicates the number of external libraries utilized by the test
Covered Lines Churn	This is an h-index that captures the changes in lines of code covered in previous periods of 5, 10, 25, 50, 75, 100, 500, and 10,000 commits. In this context, 'h' signifies that at least 'h' lines were modified at least 'h' times during that specific period

- Two *listwise* algorithms: ListNet [39] and LambdaMART [53] for the *listwise* approach.

Different methods rely on different data paradigms, so FLAKYRANK will adjust the data paradigm according to the situation. When integrating the *pointwise* approach, FLAKYRANK directly utilizes the augmented data to train the ranking model. When integrating the *pairwise* approach, FLAKYRANK constructs four types of pairs for FLAKYRANK: F-F (labeled as 0), F-NF (labeled as 1), NF-F (labeled as -1), and NF-NF (labeled as 0). Where, "F" and "NF" respectively represent the flaky tests and non-flaky tests in the processed dataset. The listwise method places greater emphasis on the overall optimality of the ranking results, which treats the entire sequence as a single sample and directly optimizes the overall ranking results. The ideal outcome is to arrange Flaky tests as much as possible at the top of the list. When integrating the *listwise* approach, FLAKYRANK restructure the processed data. To meet the input requirements of the listwise method, inspired by the information gain (IG) [26], we categorize the labels of the flaky tests into five rating levels based on the feature vectors of these samples: Extremely, Significantly, Relatively, Slightly, and Marginal, corresponding to labels 5, 4, 3, 2, and 1, respectively. The purpose of restructuring the data is to meet the input data format requirements of Listwise, enabling optimization of the overall ranking results for Flaky tests.

The restructured data is then used by FLAKYRANK to train the ranking model. For querying flaky tests in test suites, the features of the flaky tests are inputted into the trained ranking model. The model then returns a ranking list of flaky tests as the prediction results, where we examine the ranking list to predict which tests are flaky.

IV. EXPERIMENTS

A. Research Questions

In our study, we aim to analyze the following research questions:

- **RQ1a: What is the best LTR algorithm for FLAKYRANK when ranking candidate flaky tests?** In this RQ, we investigate various FLAKYRANK configurations with assembled LTR algorithms, including *pointwise*, *pairwise*, and *listwise* methods, and find the most effective FLAKYRANK setup for ranking flaky tests.
- **RQ1b: How does FLAKYRANK perform in ranking flaky tests?** Building upon RQ1a, we delve further into the investigation of the performance of FLAKYRANK across different projects. Specifically, we conducted a comprehensive analysis of the ranking results of FLAKYRANK for each project in predicting flaky tests.
- **RQ2: How effective is FLAKYRANK on predicting flaky test as compared with the state-of-the-art flaky test classifiers?** In this RQ, we compare the performance of FLAKYRANK with two flaky test predictors, FlakeFlagger [26] and Vocabulary-based [18] approaches.

B. Experiment Design

Dataset. To evaluate FLAKYRANK on predicting flaky tests, we used a publicly available dataset for flaky test cases, FlakeFlagger dataset[2]. The FlakeFlagger dataset is a new dataset of flaky tests created by exhaustively executing the test suites of 24 projects 10,000 times each, which is significantly more than prior work. The dataset encompasses the source code of these projects and categorizes the tests cases into "Flaky" and "Non-Flaky", with labels assigned as 1 and 0, respectively. Since one project of FlakeFlagger does not have flaky tests, our study uses 23 projects existing flaky test to evaluate FLAKYRANK.

Steps and Parameters. Our experiments can be divided into three steps: feature representation, data processing and candidate flaky tests ranking. Feature representation involves feature extraction from test suites of dataset, including test smells and behavior features. We practically rerun the test suites to extract these features and inspect them for any missing or unexpected issues, ensuring the rigor of our experiment results. We conduct our experiments using *k*-fold cross-validation [54].

The dataset has only 811 flaky tests, accounting for approximately 3.6% of the total test cases. In the dataset, ten projects had fewer than ten flaky tests, four projects had only one, and one project had no observed flaky tests at all. The LTR model trained on such an imbalanced dataset, number of flaky tests far less than non flaky tests, tend to be biased toward the non flaky test class. To acquire an augmented (i.e., flaky and non-flaky class-balanced) data, we employed GANs to augment the small sample sizes. Specifically, the flaky tests of the train set $[f_1, f_2...f_n]$, after injecting random noise vector, is inputted into the generating network G, which

[2]https://github.com/AlshammariA/FlakeFlagger/tree/main/flakiness-predicter

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

generates synthetic flaky tests $[l_1, l_2...l_n]$. The synthetic data vector $[l_1, l_2...l_n]$ and real flaky tests are concatenated and feed into the discriminating network D. The synthetic flaky tests are assigned a score, while raw real data are assigned a score of 1. The difference between these two scores is trained through loss back-propagation. After GANs training, FLAKYRANK learns a GANs model which generates synthetic flaky tests until obtain a class-balanced dataset for predicting flaky tests.

FLAKYRANK is a ranking framework that can integrate different LTR [55] algorithms (i.e, *pointwise*, *pairwise*, and *listwise*). FLAKYRANK will adjust the augmented data paradigm according to the LTR methods for training the ranking model. For the *pointwise* LTR algorithms, the class-balanced data will be directly used to train ranking model. For the *pairwise* LTR algorithms, FLAKYRANK constructs four types pair documents (i.e., a series of combination of flaky tests and non-flaky tests). For the *listwise* LTR algorithms, we first reassign labels to the flaky test cases in the class-balanced dataset into five levels based on the range of Information Gain [26], [44]. These levels correspond to values of the label 5, 4, 3, 2, and 1. LambdaMART belonging to the *listwise* LTR algorithms, the number of trees is a key parameter for LambdaMART. We evaluate the performance of LambdaMART under the different number of trees.

For querying flaky tests, the processed data in the form of documents is fed into the trained LTR model, which then outputs a ranked list of candidate flaky tests.

Evaluation Metrics. To evaluate FLAKYRANK, we adopt some widely-used metrics: Mean Average Precision (MAP) [56], Mean Reciprocal Rank (MRR) [57], [58], Normalized Discounted Cumulative Gain (NDCG) [59], [60], Precison, Recall, F1-score. Following the conventional strategy, we use MAP@5 for MAP, and NDCG@3, NDCG@5 and NDCG@10 for NDCG. Their definitions are as follows:

MAP@k assesses ranking model performance by measuring the average retrieval accuracy of the top k retrieved items across all queries. It's a valuable metric for evaluating ranking models. The formula for MAP@K is:

$$MAP@k = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\min(k, M_i)}\sum_{j=1}^{K}P_{i,j}\cdot rel_{i,j} \quad (3)$$

Where M_i is the number of items retrieved for the *i*-th query, $P_{i,j}$ is the precision of the model at the *j*-th position for the *i*-th query, and $rel_{i,j}$ is a binary indicator denoting the relevance of the *j*-th retrieved item for the *i*-th query.

MRR evaluates ranking systems by measuring the position of the first relevant document within search results. The formula for calculating MRR is as follows:

$$MRR = \frac{1}{|Q|}\sum_{i=1}^{|Q|}\frac{1}{rank_i} \quad (4)$$

Where Q represents the total number of questions in the evaluation set, and $rank_i$ represents the position of the first correct answer predicted for the i-th question Q_i within the answer set.

TABLE III
RESULTS OF FLAKYRANK USING DIFFERENT LTR ALGORITHMS FOR PREDICTING FLAKY TESTS.

Approach		MAP@5	NDCG@3	NDCG@5	NDCG@10
Pointwise	RF	0.64	0.52	0.50	0.49
	KNN	0.42	0.40	0.35	0.30
	SVM	0.40	0.36	0.32	0.30
	MLP	0.36	0.30	0.27	0.23
Pairwise	Ranknet	0.20	0.16	0.16	0.14
	SVMRank	0.32	0.29	0.28	0.26
Listwise	Listnet	0.56	0.46	0.44	0.41
	LambdaMART	0.66	0.55	0.52	0.51

NDCG@k: NDCG@k is the ratio of the Discounted Cumulative Gain (DCG@k) achieved when ranking the top k results to the Ideal Discounted Cumulative Gain (IDCG@k) under ideal circumstances. The formula for calculating NDCG@k is as follows:

$$NDCG@k = \frac{DCG@k}{IDCG@k} \quad (5)$$

DCG@k measures the quality of a ranked list, considering item relevance and their order. The formula for DCG@k is as follows:

$$DCG@k = \sum_{i=1}^{k}\frac{2^{rel_i}-1}{log_i(i+1)} \quad (6)$$

Where rel_i presents the relevance of the item at position i in the list. IDCG@k assumes that the relevance of the top k items in the ranking is arranged in the ideal way.

Precision is the proportion of true positive samples in positive prediction of a model, which measures the accuracy of positive predictions made by the model, which is formally defined by Eq. 7.

$$Precision = \frac{\left|T_f \bigcap T_f^{'}\right|}{|T_f|} \quad (7)$$

Where T_f refers to the set of flaky tests cases (i.e., the ground truth), and $T_f^{'}$ means the set of flaky test cases identified by ranking method.

Recall measures the ability of a model to correctly identify all relevant instances out of all the relevant instances available, which is formally defined by Eq. 8.

$$Recall = \frac{\left|T_f \bigcap T_f^{'}\right|}{\left|T_f^{'}\right|} \quad (8)$$

F1-score is the harmonic mean of precision and recall. It provides a balanced evaluation of a model's performance, considering both false positives and false negatives, which is formally defined by Eq. 9.

$$F1\text{-}score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (9)$$

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

C. Results

1) RQ1a Results: Table III shows the values of MAP@5, NDCG@3, NDCG@5, and NDCG@10 for FLAKYRANK with the *pointwise* methods, the *pairwise* methods , and the *listwise* methods to rank flaky tests. From which we can observe that in the *pointwise* methods, RF outperforms KNN, SVM, and MLP, in the *pairwise* methods, Ranknet outperforms SVMRank, and in the *listwise* methods, LambdaMART performs better than Listnet.

Specifically, in the *pointwise* methods, RF demonstrates higher accuracy compared to the other three algorithms. RF achieves a MAP@5 of 0.64, which is at least 34% higher than the other three methods. Additionally, NDCG@3, NDCG@5, and NDCG@10 scores are also higher for RF compared to the other three algorithms.

In the *pairwise* methods, SVMRank achieves higher precision than Ranknet. The results can be attributed to the characteristics of the flaky test dataset, where SVMRank excels in datasets with clear separation boundaries. The objective of SVMRank is to find a hyperplane that maximizes the margin between categories, which may make it more proficient at capturing these patterns compared to Ranknet.

In the *listwise* methods, LambdaMART has an MAP@5 of 0.66, with NDCG@3, NDCG@5, and NDCG@10 values of 0.55, 0.52, and 0.51, respectively. We can find that LambdaMART works better than ListNet in ranking flaky tests, outperforming by 15%, 16%, 15%, and 19% in MAP@5, NDCG@3, NDCG@5, and NDCG@10, respectively. LambdaMART performs better in ranking flaky tests.

NDCG@5 considers both relevance and ranking order, penalizes irrelevant results, is standardized for fair comparisons, and is sensitive to subtle ranking changes. Fig. 4 presents the distribution of NDCG@5 for each project, with the optimal methods in the *pointwise* (RF), *pairwise* (SVMRank), and *listwise* (LambdaMART) categories within the FlakeFlagger dataset. The blue, yellow, and red bars in Fig. 4 correspond to the NDCG@5 values of FLAKYRANK using RF, SVMRank, and LambdaMART, respectively. The results show that LambdaMART is the most effective among LTR approaches on 47% (11/23) of the projects, achieving an NDCG@5 score exceeding 0.6 on 9 projects, which has achieved the best results in ranking flaky tests on the FlakeFlagger dataset to our knowledge.

As shown in Fig. 4, we observed that LambdaMART's higher effectiveness in ranking flaky tests may be attributed to its ability to consider global correlations within the ranking list and its utilization of the reclassified data. Compared to LambdaMART, RF has the same NDCG@5 score on 4 projects and the scores of 11 projects are lower. The difference between LambdaMART and RF is slight. SVMRank ranks flaky tests relatively poorly, with an NDCG@5 score lower than the other two methods on most of projects.

TABLE IV
THE RESULTS OF FLAKYRANK WITH LAMBDAMART

Project	FLAKYRANK with LambdaMART				
	MAP@5	MRR	NDCG@3	NDCG@5	NDCG@10
achilles	1	1	0.3	0.41	0.41
activiti	1	1	0	0	0
alluxio	1	1	1	1	1
ambari	0.91	1	0.53	0.66	0.78
assertj-core	0	0	0	0	0
commons-exec	0	0	0	0	0
elastic-job-lite	1	1	0.47	0.47	0.47
handlebars.java	0	0	0	0	0
hbase	1	1	1	1	1
hector	1	1	1	1	0.86
http-request	1	1	0.7	0.79	0.64
httpcore	0.5	0.5	0.23	0.17	0.33
incubator-dubbo	0.89	1	0.47	0.34	0.37
java-websocket	1	1	0.53	0.53	0.69
logback	0	0	0	0	0
ninja	0	0	0	0	0
okhttp	0.23	0.04	1	0.85	0.55
orbit	0	0.04	0	0	0
spring-boot	1	1	1	0.82	0.75
undertow	0.87	1	0.7	0.51	0.41
wildfly	1	1	0.3	0.21	0.29
wro4j	1	1	0.77	0.83	0.76
zxing	1	1	1	1	1
Overall	0.66	0.7	0.55	0.52	0.51

Summary for RQ1a: In RQ1a, we investigate the performance of FLAKYRANK with eight different LTR algorithms in ranking flaky tests. Based on the above results, we can conclude that (1) some LTR models have shown promising result in ranking flaky tests effectively; (2) compared to the seven LTR algorithms, LambdaMART achieves better result in terms of MAP, MRR, and NDCG.

2) RQ1b Results: FLAKYRANK with LambdaMART yields the best performance in ranking flaky tests. The number of trees is a main parameter for LambdaMART. To assess the impact of different tree numbers on LambdaMART, we fine-tune the number of trees and calculate the corresponding Area Under the Curve (AUC) values. Fig. 5 presents the AUC values for LambdaMART with different numbers of trees. From the figure, we can observe that as the number of trees increases, the AUC value also increases, while maintaining stability beyond a certain threshold. We use the threshold of 90 as the number of trees for FLAKYRANK with LambdaMART algorithm.

Table IV shows the ranking results (in terms of MAP, MRR and NDCG) of FLAKYRANK with LambdaMART predicting flaky tests on every project of the FlakeFlagger dataset. FLAKYRANK assembling LambdaMART algorithm has shown promising result in ranking flaky tests. MAP is the overall precision score of ranking results, which serves as a comprehensive evaluation of the ranking model's performance. As shown in Table IV, we can observe that 65% (15/23) of the projects have MAP@5 values exceeding 0.5, which indicates that FLAKYRANK, based on LambdaMART, performs surprisingly well on most projects. Additionally, we found that 6 projects (i.e., assertj-core, commons-exec,

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

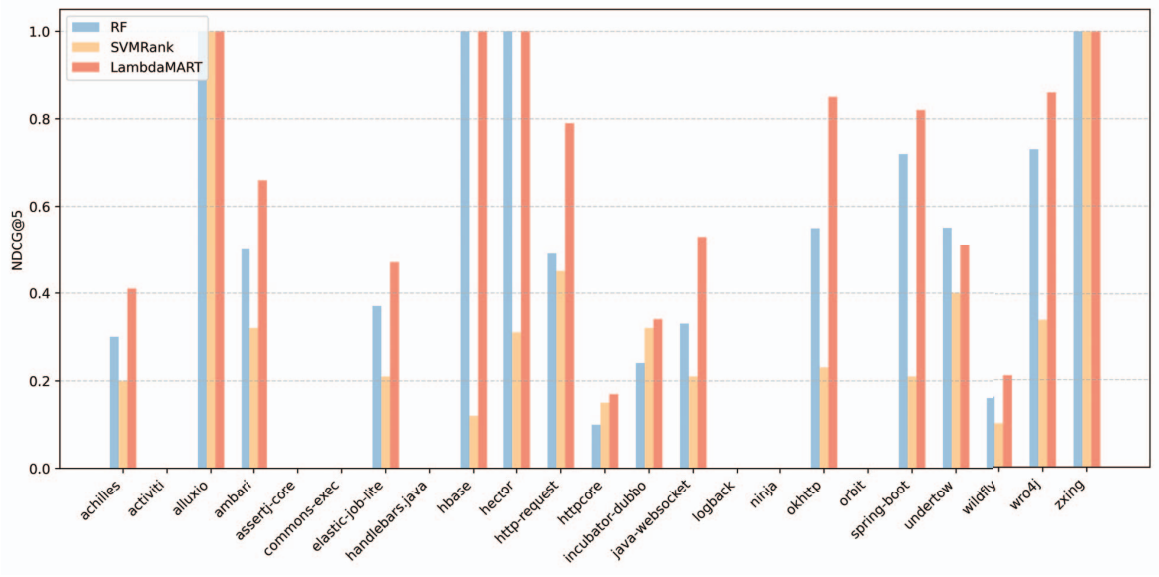


Fig. 4. FLAKYRANK with LambdaMART in ranking flaky tests NDCG@5 for each project of FlakeFagger dataset.

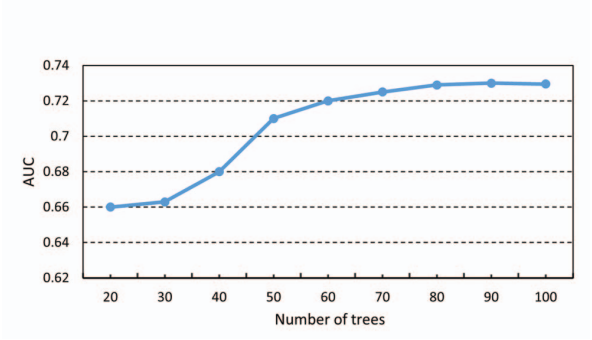


Fig. 5. The AUC values of FLAKYRANK for identification of flaky test with different tree numbers.

handlebars.java, logback, ninja, and orbit) have a MAP@5 of 0. The phenomenon may be attributed to the limited number of flaky tests, with only one in assertj-core, ninja, and commons-exec.

As shown in Table IV, we observed that 65% (15/23) of the projects achieved an MRR value of 1. In other words, for these 15 projects, the top-ranked prediction reliably corresponds to a flaky test, providing developers with a dependable result of identifying flaky tests. One project attained an MRR value of 0.5, implying that the flaky test prediction is in the second position. Additionally, five projects recorded an MRR value of 0, suggesting that, among the selected projects, no flaky tests were identified among the top-ranked samples.

NDCG serves as a criterion for assessing the quality of rankings and is used to evaluate the effectiveness of the ranking results produced by FLAKYRANK. As shown in Table IV, we observe that NDCG@3 values for five projects (Alluxio, HBase, OkHttp, Hector, and ZXing) are all 1. Similarly, NDCG@5 values for four projects (Alluxio, HBase, Hector, and ZXing) are also 1, and NDCG@10 values for three projects (Alluxio, HBase, and ZXing) are 1. There are 11 projects with NDCG@3 and NDCG@5 values exceeding 0.5, as well as 10 projects with NDCG@10 values exceeding 0.5, indicating that the quality of the ranking result sequences predicted by FLAKYRANK is outstanding for these projects.

The overall values of MAP@5, MRR, NDCG@3, NDCG@5, and NDCG@10 are 0.66, 0.7, 0.55, 0.52, and 0.51, respectively, suggesting that FLAKYRANK performs well in predicting flaky tests. The performance may be attributed to the listwise approach's emphasis on the relationship between entire samples and queries.

> *Summary for RQ1b:* In RQ1b, we evaluated the performance of FLAKYRANK with LambdaMART model on ranking flaky test cases. We can conclude that LambdaMART shows a satisfactory ranking performance in the majority of projects (13 out of 23) within the FlakeFlagger dataset.

3) RQ2 Results: To effectively compare the performance of identifying flaky tests with other two methods, we used FLAKYRANK with LambdaMART, i.e., our best-performing configuration, to predict flaky tests. FLAKYRANK will generate a ranked list that represents the potential likelihood for each test sample within the test suite, indicating whether it's a flaky test. The results are shown in Table V, from which we can find that FLAKYRANK with LambdaMART algorithm outperforms both FlakeFlagger and Vocabulary-based [18] approaches.

As shown in Table V, the precision exceeds 50% in

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RESULTS OF FLAKYRANK COMPARED TO FLAKEFLAGGER AND VOCABULARY-BASED APPROACH.

Project	FLAKYRANK			FlakeFlagger			Vocabulary-based Approach		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
achilles	100%	50%	67%	40%	50%	44%	0%	0%	0%
activiti	50%	3%	6%	19%	31%	24%	2%	38%	4%
alluxio	100%	100%	100%	100%	100%	100%	91%	93%	92%
ambari	100%	87%	93%	94%	90%	92%	23%	69%	34%
assertj-core	0%	0%	0%	0%	0%	0%	0%	0%	0%
commons-exec	0%	0%	0%	0%	0%	0%	5%	100%	10%
elastic-job-lite	100%	33%	50%	0%	0%	0%	0%	0%	0%
handlebars.java	0%	0%	0%	0%	0%	0%	0%	0%	0%
hbase	86%	93%	89%	80%	89%	84%	80%	90%	84%
hector	85%	88%	87%	79%	91%	85%	36%	39%	38%
http-request	71%	56%	63%	67%	67%	67%	16%	86%	27%
httpcore	53%	41%	46%	38%	64%	47%	4%	73%	8%
incubator-dubbo	83%	26%	40%	19%	42%	26%	1%	58%	3%
java-websocket	100%	83%	90%	95%	83%	88%	24%	100%	38%
logback	67%	9%	17%	15%	14%	14%	4%	45%	7%
ninja	0%	0%	0%	0%	0%	0%	0%	0%	0%
okhttp	48%	24%	32%	25%	52%	33%	15%	79%	25%
orbit	100%	43%	60%	11%	14%	12%	16%	86%	27%
spring-boot	97%	82%	89%	90%	87%	89%	16%	84%	27%
undertow	50%	29%	36%	20%	29%	24%	9%	86%	16%
wildfly	100%	43%	61%	29%	48%	36%	3%	87%	7%
wro4j	100%	69%	81%	67%	25%	36%	2%	12%	3%
zxing	100%	100%	100%	0%	0%	0%	1%	50%	1%
Total	82%	68%	74%	60%	74%	66%	11%	72%	19%

69% (16/23) of the projects. Our approach achieves a 36% higher precision than FlakeFlagger [26] on total projects. Four projects (i.e., ninja, assertj-core, commons-exec, and handlebars.java) have values of 0 for precision, recall, and F1-score. The phenomenon can be attributed to the limited number of flaky test samples in these four projects. Further, 39% (9 /23) of the projects have a recall value exceeding 50%. The total projects for precision, recall, and F1-score across is 82%, 68%, and 74%, respectively, which shows that FLAKYRANK is a promising direction for advancing flaky test cases detection methods.

Summary for RQ2: In RQ2, we commpare the performance of FLAKYRANK with the FlakeFlagger and Vocabulary-based approaches. The experiment results show that (1) in the Precision performance, FLAKYRANK outperforms the two baselines by at least 36% across all projects; (2) in the F1-score performance, FLAKYRANK achieves a 12% higher F1-Score than the two baselines across all projects.

V. DISCUSSION

***The impact of GANs on* FLAKYRANK.** In software test, test flakiness [12] is a common issue, and the majority of software developers frequently encounter the challenge of dealing with flaky tests [8]. In the FlakeFlagger dataset we employed, flaky tests account for approximately 3.6% of the total tests, which is due to class imbalance [61] in the dataset. Class-imbalanced dataset can lead to biased models that favor the majority class and perform poorly on the minority class, which can have a negative impact on machine learning [62].

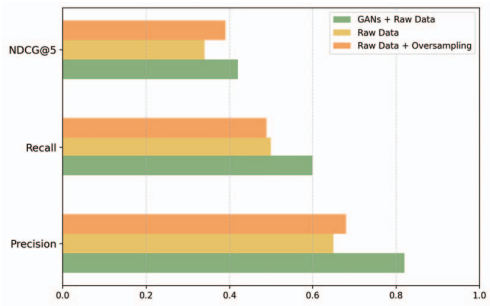


Fig. 6. Comparison of results with original, oversampled, and GAN-generated data as inputs.

To improve FLAKYRANK, we employed GANs to generate the synthetic flaky tests, which made a class-balanced dataset for train.

We conducted experiments to assess the impact of GANs on FLAKYRANK and the quality of synthetic flaky tests they generate. Three sets of input data were used for predicting flaky tests: one with raw data, another incorporating synthetic flaky tests from GANs, and a third using oversampling [63]. Previous sections explored the impact of FLAKYRANK with various LTR methods [64], with LambdaMART showing the best overall performance. To comprehensively compare the impact of GAN-enhanced data, we conducted a unified evaluation using FLAKYRANK and integrating the LambdaMART model, maintaining consistent parameters and experimental settings.

The results of predicting flaky tests with FLAKYRANK based on LambdaMART are presented in Fig. 6. As shown in Fig. 6, we can observe that employing GANs to augment data

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

is beneficial for predicting flaky tests and also contributes to the training of a ranking model. The input data with synthetic flaky tests generated by GANs outperforms the use of the original dataset and the oversampling method in terms of precision, recall, and NDCG@5. GANs enabled FLAKYRANK to identify fewer false positives and obtain a more accurate ranking model. In conclusion, the comparative experiments [65] presented in Fig. 6 illustrate the positive impact of utilizing GANs [66] with FLAKYRANK for identifying flaky tests. The introduction of synthetic flaky tests generated by GANs not only enhances precision and recall but also contributes to the training of a more accurate ranking model. These results support the use of GANs to augment data when predicting flaky tests, leading to more reliable and precise outcomes in the field of software test.

Threats to validity. There are several threats to the validity of this study. Some follow the nature of the data used while others follow the design decisions. The dataset we used, FlakeFlagger, can not guarantee that all flaky tests of these projects were labeled flaky. Further, some non-flaky tests is flaky tests in practice, which leads to false positive and false negative alarm in our experiments. To mitigate this impact, we will use alternatives datasets to validate our approach in the future. While the evaluation results are supported by carefully designed experiments, better results could be achieved with a larger dataset and labels that are more precise and closely related to the content of the test cases. Threats to construct validity lie in the measurement methods used. We employ multiple metrics widely used to evaluate both flaky tests prediction and machine learning models and conduct experiments under 10-fold cross-validation to enhance generalizability. Additionally, we used Ubuntu-20.04 as operating system and adopted widely-used jdk-1.8 as jdk version. Some flaky tests might not have been identified due to the platform limitations.

VI. RELATED WORK

Flaky tests are tests that yield non-deterministic results: they may pass or fail intermittently even when the underlying code remains unaltered [8]. For developers, encountering flaky tests during the process of regression testing can erode their confidence in the reliability of the entire test suite [67]. Flaky tests presents significant challenges for both developers and researchers alike [5]. Furthermore, the presence of flaky tests [25] can introduce uncertainty into the software development process, making it difficult to differentiate between real code issues and false test failures. Which can impede the debugging and troubleshooting processes, slowing down the development cycle.

The causes of flaky tests. Study [8] has been conducted and categorize the factors contributing to flaky tests in typical software projects, including a wide range of platforms. Timing dependencies related to asynchronous calls are a significant factor in previous studies [5], [16], [19]. Tests can be affected by timing issues when dealing with asynchronous operations, leading to instability. Platform dependency is another prominent cause. Concurrency-related issues, such as data races and deadlocks, are frequently observed sources of test instability. Additionally, test order dependencies can impact test stability, as specific execution sequences may unexpectedly lead to test failures [5], [12].

The insights to predict flaky tests. Many studies [25], [26] have proposed different approaches and techniques for identifying flaky tests. Repeatedly executing these tests is the most straightforward method for detecting flaky tests [24]. However, the approach is very time-consuming, and it's challenging to determine the number of repetitions. Too few repetitions may result in missing elusive flaky tests, while too many can increase the runtime costs [8], [13]. Therefore, more efficient methods have been proposed. These methods can be divided into two categories: those that require test execution and those that do not require test execution to predict flaky tests. Some of the former methods involve adding certain plugins to tests for identification, but this approach is only applicable to specific cases of flaky tests, such as asynchronous waiting category of flaky tests [12], [68]. Additionally, there are techniques that leverage specific characteristics to identify flaky tests, such as differential coverage [24], deterministic implementation of non-deterministic specifications (ADINSs) [10], and manually specified determinism constraints [69]. Some studies [18] have introduced approaches to predict flaky tests using machine learning, pattern recognition, and large language models [70]. These methods employ static analysis to identify common syntactic code patterns related to errors in test cases [71], identifiers and keywords [72] , vocabulary-based flaky tests features [73], test smell features [43], and more.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose FLAKYRANK: an augmented learning to rank approach for predicting flaky tests. The basic idea of FLAKYRANK is that we treat predicting flaky tests as an information retrieval task. To embrace and implement the idea, we first use generative adversarial networks to augment the dataset for the prediction model, then leverage learning to rank models to acquire a ranking of flaky tests via the behavior features of test cases, finally predict flaky tests using the ranking list. To evaluate our approach FLAKYRANK, we conduct large-scale experiments on 23 benchmark projects. The results show that different learning to rank models have different impact on flaky test prediction, and FLAKYRANK with LambdaMART algorithm performs the best. In addition, the results also show that FLAKYRANK performs better than the state-of-the-art flaky test predictors (i.e., FlakeFlagger and Vocabulary-based approaches.)

ACKNOWLEDGMENT

This work is partially supported by the National Natural Science Foundation of China (No. 62272072), the Major Key Project of PCL (No. PCL2021A06), the Chongqing Technology Innovation and Application Development Special Project (CSTB2023TIAD-STX0029) and the Fundamental Research Funds for the Central Universities (No. 2022CDJDX-005).

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] S. Anand, E. K. Burke, T. Y. Chen, J. Clark, M. B. Cohen, W. Grieskamp, M. Harman, M. J. Harrold, P. McMinn, A. Bertolino *et al.*, "An orchestrated survey of methodologies for automated software test case generation," *Journal of systems and software*, vol. 86, no. 8, pp. 1978–2001, 2013.

[2] P. McMinn, "Search-based software test data generation: a survey," *Software testing, Verification and reliability*, vol. 14, no. 2, pp. 105–156, 2004.

[3] B. Korel, "Automated software test data generation," *IEEE Transactions on software engineering*, vol. 16, no. 8, pp. 870–879, 1990.

[4] J. Kasurinen, O. Taipale, and K. Smolander, "Software test automation in practice: empirical observations," *Advances in Software Engineering*, vol. 2010, 2010.

[5] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2019, pp. 830–840.

[6] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 101–111.

[7] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "idflakies: A framework for detecting and partially classifying flaky tests," in *2019 12th ieee conference on software testing, validation and verification (icst)*. IEEE, 2019, pp. 312–322.

[8] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, pp. 1–74, 2021.

[9] C. Ziftci and D. Cavalcanti, "De-flake your tests: Automatically locating root causes of flaky tests in code at google," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 736–745.

[10] A. Shi, J. Bell, and D. Marinov, "Mitigating the effects of flaky tests on mutation testing," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 112–122.

[11] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming google-scale continuous testing," in *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*. IEEE, 2017, pp. 233–242.

[12] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, 2014, pp. 643–653.

[13] W. Lam, S. Winter, A. Astorga, V. Stodden, and D. Marinov, "Understanding reproducibility and characteristics of flaky tests through test reruns in java projects," in *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 2020, pp. 403–413.

[14] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*, 2016, pp. 426–437.

[15] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. D. Ernst, and D. Notkin, "Empirically revisiting the test independence assumption," in *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, 2014, pp. 385–396.

[16] A. Vahabzadeh, A. M. Fard, and A. Mesbah, "An empirical study of bugs in test code," in *2015 IEEE international conference on software maintenance and evolution (ICSME)*. IEEE, 2015, pp. 101–110.

[17] T. M. King, D. Santiago, J. Phillips, and P. J. Clarke, "Towards a bayesian network model for predicting flaky automated tests," in *2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 2018, pp. 100–107.

[18] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *Proceedings of the 17th International Conference on Mining Software Repositories*, 2020, pp. 492–502.

[19] W. Lam, K. Muşlu, H. Sajnani, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, 2020, pp. 1471–1482.

[20] J. Micco, "The state of continuous integration testing@ google," 2017.

[21] S. Dutta, A. Shi, R. Choudhary, Z. Zhang, A. Jain, and S. Misailovic, "Detecting flaky tests in probabilistic and machine learning applications," in *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, 2020, pp. 211–224.

[22] A. Shi, A. Gyori, O. Legunsen, and D. Marinov, "Detecting assumptions on deterministic implementations of non-deterministic specifications," in *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2016, pp. 80–90.

[23] D. Silva, L. Teixeira, and M. d'Amorim, "Shake it! detecting flaky tests caused by concurrency with shaker," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020, pp. 301–311.

[24] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "Deflaker: Automatically detecting flaky tests," in *Proceedings of the 40th international conference on software engineering*, 2018, pp. 433–444.

[25] R. Verdecchia, E. Cruciani, B. Miranda, and A. Bertolino, "Know you neighbor: Fast static prediction of test flakiness," *IEEE Access*, vol. 9, pp. 76 119–76 134, 2021.

[26] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests. in 2021 ieee/acm 43rd international conference on software engineering (icse)," *IEEE, IEEE, Madrid, Spain*, pp. 1572–1584, 2021.

[27] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[28] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, U. Erlingsson *et al.*, "Extracting training data from large language models," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2633–2650.

[29] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative adversarial networks: An overview," *IEEE signal processing magazine*, vol. 35, no. 1, pp. 53–65, 2018.

[30] K. Wang, C. Gou, Y. Duan, Y. Lin, X. Zheng, and F.-Y. Wang, "Generative adversarial networks: introduction and outlook," *IEEE/CAA Journal of Automatica Sinica*, vol. 4, no. 4, pp. 588–598, 2017.

[31] M.-Y. Liu and O. Tuzel, "Coupled generative adversarial networks," *Advances in neural information processing systems*, vol. 29, 2016.

[32] O. Chapelle, Y. Chang, and T.-Y. Liu, "Future directions in learning to rank," in *Proceedings of the Learning to Rank Challenge*. PMLR, 2011, pp. 91–100.

[33] J. Gui, Z. Sun, Y. Wen, D. Tao, and J. Ye, "A review on generative adversarial networks: Algorithms, theory, and applications," *IEEE transactions on knowledge and data engineering*, vol. 35, no. 4, pp. 3313–3332, 2021.

[34] M. Pavan Kumar and P. Jayagopal, "Generative adversarial networks: a survey on applications and challenges," *International Journal of Multimedia Information Retrieval*, vol. 10, no. 1, pp. 1–24, 2021.

[35] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4401–4410.

[36] L. J. Ratliff, S. A. Burden, and S. S. Sastry, "Characterization and computation of local nash equilibria in continuous games," in *2013 51st Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. IEEE, 2013, pp. 917–924.

[37] X. Yu, H. Dai, L. Li, X. Gu, J. W. Keung, K. E. Bennin, F. Li, and J. Liu, "Finding the best learning to rank algorithms for effort-aware defect prediction," *Information and Software Technology*, p. 107165, 2023.

[38] H. Wu, Q. Liang, W. Zhang, Q. Zou, A. E.-L. Hesham, and B. Liu, "ilncda-ltr: Identification of lncrna-disease associations by learning to rank," *Computers in Biology and Medicine*, vol. 146, p. 105605, 2022.

[39] Z. Cao, T. Qin, T.-Y. Liu, M.-F. Tsai, and H. Li, "Learning to rank: from pairwise approach to listwise approach," in *Proceedings of the 24th international conference on Machine learning*, 2007, pp. 129–136.

[40] F. Xia, T.-Y. Liu, J. Wang, W. Zhang, and H. Li, "Listwise approach to learning to rank: theory and algorithm," in *Proceedings of the 25th international conference on Machine learning*, 2008, pp. 1192–1199.

[41] A. Ahmad, O. Leifler, and K. Sandahl, "Empirical analysis of factors and their effect on test flakiness-practitioners' perceptions," *arXiv preprint arXiv:1906.00673*, 2019.

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.

[42] R. Ramler, M. Moser, and J. Pichler, "Automated static analysis of unit test code," in *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*, vol. 2. IEEE, 2016, pp. 25–28.

[43] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "Flakeflagger: Predicting flakiness without rerunning tests," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1572–1584.

[44] B. Camara, M. Silva, A. Endo, and S. Vergilio, "On the use of test smells for prediction of flaky tests," in *Proceedings of the 6th Brazilian Symposium on Systematic and Automated Software Testing*, 2021, pp. 46–54.

[45] Y. Lei, T. Wen, H. Xie, L. Fu, C. Liu, L. Xu, and H. Sun, "Mitigating the effect of class imbalance in fault localization using context-aware generative adversarial network," *arXiv preprint arXiv:2303.06644*, 2023.

[46] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, A. Ihara, and K. Matsumoto, "The impact of mislabelling on the performance and interpretation of defect prediction models," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1. IEEE, 2015, pp. 812–823.

[47] A. Liaw, M. Wiener *et al.*, "Classification and regression by randomforest," *R news*, vol. 2, no. 3, pp. 18–22, 2002.

[48] C.-C. Chang and C.-J. Lin, "Libsvm: a library for support vector machines," *ACM transactions on intelligent systems and technology (TIST)*, vol. 2, no. 3, pp. 1–27, 2011.

[49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[50] S. Sun and R. Huang, "An adaptive k-nearest neighbor algorithm," in *2010 seventh international conference on fuzzy systems and knowledge discovery*, vol. 1. IEEE, 2010, pp. 91–94.

[51] C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, and G. Hullender, "Learning to rank using gradient descent," in *Proceedings of the 22nd international conference on Machine learning*, 2005, pp. 89–96.

[52] T. Joachims, "Training linear svms in linear time," in *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2006, pp. 217–226.

[53] C. J. Burges, "From ranknet to lambdarank to lambdamart: An overview," *Learning*, vol. 11, no. 23-581, p. 81, 2010.

[54] R. Kohavi *et al.*, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Ijcai*, vol. 14, no. 2. Montreal, Canada, 1995, pp. 1137–1145.

[55] H. Valizadegan, R. Jin, R. Zhang, and J. Mao, "Learning to rank by optimizing ndcg measure," *Advances in neural information processing systems*, vol. 22, 2009.

[56] C. Pei, Y. Zhang, Y. Zhang, F. Sun, X. Lin, H. Sun, J. Wu, P. Jiang, J. Ge, W. Ou *et al.*, "Personalized re-ranking for recommendation," in *Proceedings of the 13th ACM conference on recommender systems*, 2019, pp. 3–11.

[57] G. Salton and C. Buckley, "Improving retrieval performance by relevance feedback," *Journal of the American society for information science*, vol. 41, no. 4, pp. 288–297, 1990.

[58] E. M. Voorhees *et al.*, "The trec-8 question answering track report." in *Trec*, vol. 99, 1999, pp. 77–82.

[59] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of ir techniques," *ACM Transactions on Information Systems (TOIS)*, vol. 20, no. 4, pp. 422–446, 2002.

[60] T. Sakai, "Alternatives to bpref," in *Proceedings of the 30th annual international ACM SIGIR conference on Research and development in information retrieval*, 2007, pp. 71–78.

[61] N. Japkowicz and S. Stephen, "The class imbalance problem: A systematic study," *Intelligent data analysis*, vol. 6, no. 5, pp. 429–449, 2002.

[62] M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015.

[63] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "Smote: synthetic minority over-sampling technique," *Journal of artificial intelligence research*, vol. 16, pp. 321–357, 2002.

[64] C. He, C. Wang, Y.-X. Zhong, and R.-F. Li, "A survey on learning to rank," in *2008 International Conference on Machine Learning and Cybernetics*, vol. 3. Ieee, 2008, pp. 1734–1739.

[65] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[66] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial networks," *Communications of the ACM*, vol. 63, no. 11, pp. 139–144, 2020.

[67] A. Gyori, A. Shi, F. Hariri, and D. Marinov, "Reliable testing: Detecting state-polluting tests to prevent test dependency," in *Proceedings of the 2015 international symposium on software testing and analysis*, 2015, pp. 223–233.

[68] A. T. Endo and A. Møller, "Noderacer: Event race detection for node. js applications," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 120–130.

[69] R. Mudduluru, J. Waataja, S. Millstein, and M. Ernst, "Verifying determinism in sequential programs," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 37–49.

[70] S. Fatima, H. Hemmati, and L. Briand, "Black-box prediction of flaky test fix categories using language models," *arXiv preprint arXiv:2307.00012*, 2023.

[71] S. Person and S. Elbaum, "Test analysis: Searching for faults in tests (n)," in *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2015, pp. 149–154.

[72] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in android apps," in *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2018, pp. 534–538.

[73] G. Haben, S. Habchi, M. Papadakis, M. Cordy, and Y. Le Traon, "A replication study on the usability of code vocabulary in predicting flaky tests," in *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 2021, pp. 219–229.

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 28,2026 at 15:14:55 UTC from IEEE Xplore. Restrictions apply.